EST0133 - Projeto II

Marcus Nunes

9 de Abril de 2021

Instruções

- A data limite de entrega é 05/04/2021, às 23:59, via SIGAA
- O R é o único software permitido para coleta, limpeza e análise dos dados
- Se preferir fazer o trabalho em R Markdown, utilize o arquivo `modelo.Rmd` para isso
- Renomeie-o para `NomeSobrenome.Rmd`, em que `Nome` é o seu primeiro nome e `Sobrenome` é um de seus sobrenomes
- Se preferir realizar o trabalho em outro formato que não seja R Markdown, os códigos utilizados e as respostas para as perguntas devem ser fornecidas em dois arquivos identificados como `NomeSobrenome.R` e `NomeSobrenome.pdf`
- Todos os códigos apresentados devem estar em fonte `monoespaçada`, como a `Courier New`, por exemplo
- Envie conjuntamente seus arquivos `NomeSobrenome.Rmd` (ou `NomeSobrenome.R`) e `NomeSobrenome.pdf` para avaliação em um arquivo chamado `NomeSobrenome.zip`
- Respostas em arquivos que não estejam nos formatos Rmd (ou R) e pdf não serão consideradas
- Identifique corretamente os eixos dos gráficos produzidos
- Respostas numeradas incorretamente não serão corrigidas
- Respostas com códigos supérfluos para a sua resolução, como pacotes desnecessários para as análises realizadas, terão pontuação descontada
- Não é permitido reportar resultados como capturas de tela (*screenshots*), a menos que sejam figuras
- Nos exercícios em que for necessário ajustar uma semente aleatória, rode o código `set.seed(x, kind = "Mersenne-Twister", normal.kind = "Inversion")`, em que `x` é o número da semente pedida

Parte I

O arquivo `volei.csv` traz resultados de partidas de 28.664 duplas de vôlei de praia, tanto masculinas quanto femininas. Suas variáveis são

- `circuit`: entidade organizadora da competição
- `gender`: gênero da dupla
- `year`: ano da partida
- `attacks`: número de ataques realizados
- `kills`: número de ataques realizados que resultaram em pontos
- `errors`: número de erros
- `aces`: pontos de saque
- `serve_errors`: erros de saque
- `blocks`: pontos de bloqueio
- `digs`: número de defesas bem-sucedidas
- `win`: resultado da partida

Queremos criar um modelo preditivo para o resultado da partida, sem considerar seu ano de realização.

Questão 1

(*05 pontos*) O primeiro passo será preparar o conjunto de dados para análise. Para isso, crie um objeto chamado `vb` com apenas as partidas de duplas femininas realizadas a partir de 2015 (inclusive) no circuito AVP. Remova do conjunto de dados as variáveis redundantes para a análise.

```
library(tidymodels)
```

```
## -- Attaching packages ------------------------------------- tidymodels 0.1.2 --

## v broom     0.7.6      v recipes   0.1.15
## v dials     0.0.9      v rsample   0.0.9
## v dplyr     1.0.5      v tibble    3.1.0
## v ggplot2   3.3.3      v tidyr     1.1.3
## v infer     0.5.4      v tune      0.1.3
## v modeldata 0.1.0      v workflows 0.2.2
## v parsnip   0.1.5      v yardstick 0.0.8
## v purrr     0.3.4

## -- Conflicts ---------------------------------------- tidymodels_conflicts() --
## x purrr::accumulate() masks foreach::accumulate()
## x purrr::discard()    masks scales::discard()
## x dplyr::filter()     masks stats::filter()
## x dplyr::lag()        masks stats::lag()
## x recipes::step()     masks stats::step()
## x purrr::when()       masks foreach::when()
```

```
theme_set(theme_bw())
library(ggfortify)
library(GGally)
```

```
## Registered S3 method overwritten by 'GGally':
##   method from
##   +.gg   ggplot2
```

```r
library(vip)
```

```
##
## Attaching package: 'vip'

## The following object is masked from 'package:utils':
##
##     vi
```

```r
volei <- read.csv(file = "dados/volei.csv")

vb <-
  volei %>%
  # volei feminino
  filter(gender == "W") %>%
  # anos recentes
  filter(year >= 2015) %>%
  # remover circuito
  filter(circuit == "AVP") %>%
  # remover variaveis
  select(-circuit, -gender, -year)
```

Questão 2

(*05 pontos*) Utilize a semente 5659 para criar os conjuntos de treino e teste. O conjunto de treino deve ser criado com 73% das observações.

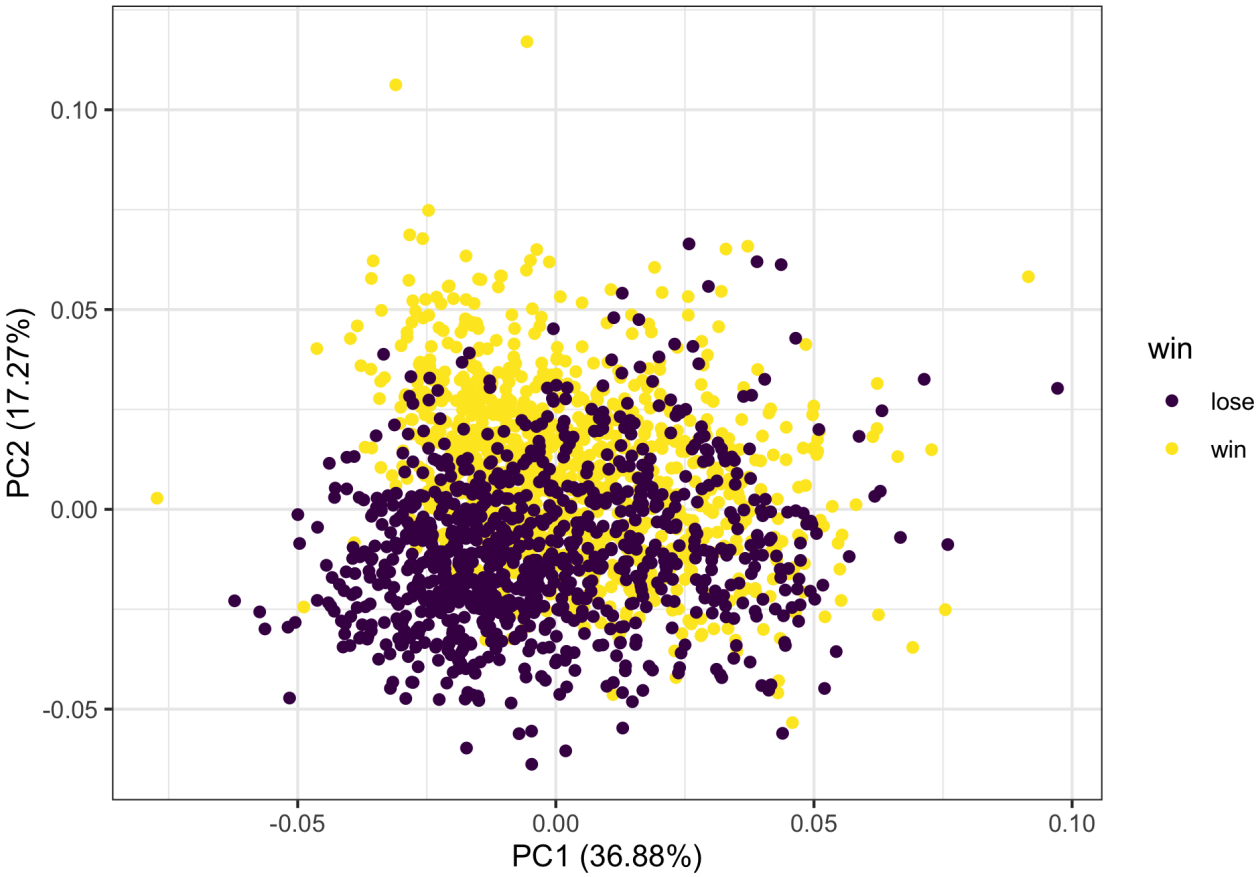
```r
# semente aleatoria

set.seed(5659, kind = "Mersenne-Twister", normal.kind = "Inversion")

# 73% dos dados como treino

vb_split <- initial_split(vb, prop = .73, strata = win)

# criar os conjuntos de dados de treino e teste

vb_treino <- training(vb_split)

vb_teste  <- testing(vb_split)
```

Questão 3

(*05 pontos*) Visualize a PCA do conjunto de treino em um gráfico de dispersão, colorindo os pontos de acordo com a variável resposta. Discuta a variabilidade captada pela PCA. Os dados aparentam ser linearmente separáveis?

```r
# pca

autoplot(prcomp(vb_treino %>% select(-win),
                center = TRUE,
                scale. = TRUE),
         data = vb_treino,
         colour = "win") +
  scale_colour_viridis_d()
```

```
## Warning: `select_()` was deprecated in dplyr 0.7.0.
## Please use `select()` instead.
```

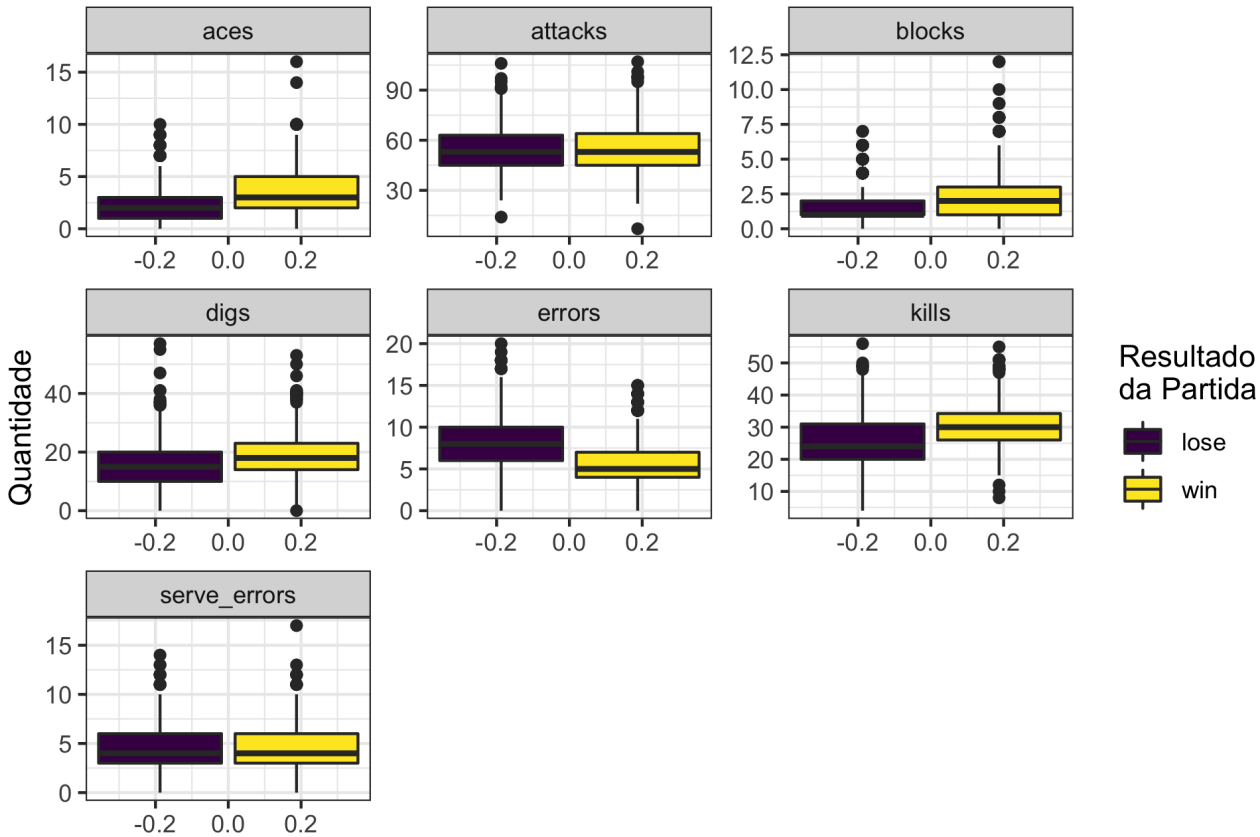


As duas primeiras componentes principais correspondem a 54.15% da variabilidade dos dados. Não é um índice considerado alto, mas também não é algo a se desprezar. Seriam necessárias 4 componentes principais para obtermos pelo menos 80% da variância dos dados. Além disso, esse dados não parecem ser linearmente separáveis.

Questão 4

(*05 pontos*) Crie boxplots comparando os valores das variáveis preditoras que permaneceram no conjunto de dados de acordo com o resultado das partidas. Na sua opinião, quais poderiam ser as três variáveis preditoras que melhor discriminam o resultado das partidas? Por quê?

```r
vb_treino %>%
  pivot_longer(-win) %>%
  ggplot(., aes(fill = win)) +
  geom_boxplot(aes(y = value)) +
  facet_wrap(~ name, scales = "free") +
  labs(x = "", y = "Quantidade", fill = "Resultado\nda Partida") +
  scale_fill_viridis_d()
```



Por apresentarem a maior diferença entre os níveis, as variáveis `aces`, `errors` e `kills` poderiam ser as três variáveis preditoras que melhor discriminam o resultado das partidas.

Questão 5

(*05 pontos*) Pré-processe os dados com apenas três transformações:

i) Balanceie o número de observações para cada classe da variável resposta;

ii) Deixe a média das variáveis preditoras igual a zero;

iii) Faça com que a variância das variáveis preditoras seja igual a um.

Não é necessário realizar nenhum outro tipo de pré-processamento para essa análise. Aplique as transformações nos conjuntos de treino e teste.

```r
# pre-processamento

vb_rec <-
  recipe(win ~ .,
         data = vb_treino) %>%
  # remover observacoes de modo que todos os niveis de win
  # fiquem com o mesmo numero de observacoes
  themis::step_downsample(win) %>%
  # center/scale
  step_center(-win) %>%
  step_scale(-win) %>%
  # funcao para aplicar a transformacao aos dados
  prep()
```

```
## Registered S3 methods overwritten by 'themis':
##   method                  from
##   bake.step_downsample    recipes
##   bake.step_upsample      recipes
##   prep.step_downsample    recipes
##   prep.step_upsample      recipes
##   tidy.step_downsample    recipes
##   tidy.step_upsample      recipes
##   tunable.step_downsample recipes
##   tunable.step_upsample   recipes
```

```r
# aplicar a transformacao aos dados

vb_treino_t <- juice(vb_rec)

# preparar o conjunto de teste

vb_teste_t <- bake(vb_rec,
                   new_data = vb_teste)
```

Questão 6

(*05 pontos*) Defina a validação cruzada com 10 grupos para avaliar o desempenho dos algoritmos que aplicaremos a esses dados. Utilize a semente 6472 para isso.

```r
set.seed(6472, kind = "Mersenne-Twister", normal.kind = "Inversion")

vb_treino_cv <- vfold_cv(vb_treino_t, v = 10)
```

Questão 7

(*05 pontos*) Crie grids de procura para os hiperparâmetros dos métodos SVM linear e random forest. Encontre o melhor valor de custo para o SVM entre os valores 2^{-10} e 2^{10}. Para o random forest, escolha quatro valores diferentes para `mtry` entre 1 e o máximo permitido, dois valores para `trees` entre 500 e 1000 e 5 valores para `min_n` entre 10 e 50.

```r
# definicao do tuning - svm

vb_svm_poly_tune  <-
  svm_poly(
    cost = tune(),
    degree = 1) %>%
  set_engine("kernlab") %>%
  set_mode("classification")

# grid de procura

vb_svm_poly_grid <- grid_regular(cost (range(-10, 10)),
                                 levels = 11)

# definicao do tuning - rf

vb_rf_tune <-
  rand_forest(
    mtry = tune(),
    trees = tune(),
    min_n = tune()
  ) %>%
  set_mode("classification") %>%
  set_engine("ranger", importance = "impurity")

# grid de procura

vb_rf_grid <- grid_regular(mtry(range(1, 7)),
                           trees(range(500, 1000)),
                           min_n(range(10, 50)),
                           levels = c(4, 2, 5))
```

Questão 8

(*05 pontos*) Considerando o critério da acurácia, qual dos métodos utilizados apresentou o melhor resultado no conjunto de treino?

```r
# workflow - svm

vb_svm_poly_tune_wflow <-
  workflow() %>%
  add_model(vb_svm_poly_tune) %>%
  add_formula(win ~ .)

# avaliacao do modelo - svm

vb_svm_poly_fit_tune <-
  vb_svm_poly_tune_wflow %>%
  tune_grid(
    resamples = vb_treino_cv,
    grid = vb_svm_poly_grid
  )
```

```r
# workflow - rf

vb_rf_tune_wflow <-
  workflow() %>%
  add_model(vb_rf_tune) %>%
  add_formula(win ~ .)

# avaliacao do modelo - rf

vb_rf_fit_tune <-
  vb_rf_tune_wflow %>%
  tune_grid(
    resamples = vb_treino_cv,
    grid = vb_rf_grid
  )

# melhor modelo - svm

vb_svm_poly_fit_tune %>%
  show_best("accuracy", n = 1)
```

```
## # A tibble: 1 x 7
##    cost .metric  .estimator  mean     n std_err .config
##   <dbl> <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1    64 accuracy binary     0.831    10 0.00696 Preprocessor1_Model09
```

```r
# melhor modelo - rf

vb_rf_fit_tune %>%
  show_best("accuracy", n = 1)
```

```
## # A tibble: 1 x 9
##    mtry trees min_n .metric  .estimator  mean     n std_err .config
##   <int> <int> <int> <chr>    <chr>      <dbl> <int>   <dbl> <chr>
## 1     1   500    10 accuracy binary     0.809    10 0.00988 Preprocessor1_Model~
```

De acordo com a acurácia, o SVM Linear apresentou o melhor resultado no conjunto de treino, 0.831 contra 0.809 do random forest.

Questão 9

(*05 pontos*) Houve sobreajuste em algum dos algoritmos utilizados? Justifique.

```r
# selecao e ajuste do melhor modelo - svm

vb_svm_poly_best <-
  vb_svm_poly_fit_tune %>%
  select_best("accuracy")

vb_svm_poly_final <-
  vb_svm_poly_tune_wflow %>%
```

```r
  finalize_workflow(vb_svm_poly_best)

vb_svm_poly_final <- fit(vb_svm_poly_final, vb_treino_t)

# selecao e ajuste do melhor modelo - rf

vb_rf_best <-
  vb_rf_fit_tune %>%
  select_best("accuracy")

vb_rf_final <-
  vb_rf_tune_wflow %>%
  finalize_workflow(vb_rf_best)

vb_rf_final <- fit(vb_rf_final, vb_treino_t)

# verificando sobreajuste

resultado_svm_poly <-
  vb_teste_t %>%
  bind_cols(predict(vb_svm_poly_final, vb_teste_t) %>%
              rename(predicao_svm_poly = .pred_class))

resultado_rf <-
  vb_teste_t %>%
  bind_cols(predict(vb_rf_final, vb_teste_t) %>%
              rename(predicao_rf = .pred_class))

# metricas finais no conjunto de teste

metrics(resultado_svm_poly,
        truth = win,
        estimate = predicao_svm_poly)
```

```
## # A tibble: 2 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.825
## 2 kap      binary         0.650
```

```r
metrics(resultado_rf,
        truth = win,
        estimate = predicao_rf)
```

```
## # A tibble: 2 x 3
##   .metric  .estimator .estimate
##   <chr>    <chr>          <dbl>
## 1 accuracy binary         0.825
## 2 kap      binary         0.650
```

Os valores das acurácias nos conjuntos de treino e teste são similares para ambos os algoritmos, então não temos motivos para suspeitar de sobreajuste.

Questão 10

(*05 pontos*) O resultado obtido com a modelagem definitiva é bom o suficiente, na sua opinião? Cite alguma sugestão, a ser aplicada nos dados ou na modelagem, que talvez pudesse melhorar o resultado obtido. Não é necessário implementar a sugestão, apenas comentá-la e justificá-la.

O resultado é bastante razoável. Uma acurácia igual a 0.831 não é um resultado ruim. Caso queiramos melhorar o desempenho obtido, algumas alternativas possíveis são

- refinar a grade de procura para encontrar hiperparâmetros melhores
- aplicar outras transformações nas variáveis preditoras
- utilizar mais dados, considerando partidas ocorridas antes de 2015

Parte II

O pacote `Ecdat` possui um conjunto de dados chamado `Housing`, com preços de casas no Canadá em 1987. São 546 observações e 12 colunas:

- `price`: preço de venda da casa
- `lotsize`: tamanho do terreno do imóvel
- `bedrooms`: número de quartos
- `bathrms`: número de banheiros (excluindo lavabos)
- `stories`: número de andares, excluindo porão
- `driveway`: a casa possui entrada de garagem?
- `recroom`: a casa possui sala de jogos?
- `fullbase`: a casa possui um porão finalizado?
- `gashw`: a casa utiliza gas natural para aquecimento da água?
- `airco`: a casa possui ar condicionado central?
- `garagepl`: número de vagas de garagem
- `prefarea`: a casa está localizada em área nobre?

O objetivo desse problema é prever o preço dos imóveis a partir das outras variáveis preditoras.

Questão 11

(*05 pontos*) Utilize a semente 9890 para criar os conjuntos de treino e teste. O conjunto de treino deve ser criado com 77% das observações.

```
library(tidymodels)
theme_set(theme_bw())
library(Ecdat)
```

```
## Loading required package: Ecfun
```

```
##
## Attaching package: 'Ecfun'
```

```
## The following object is masked from 'package:base':
##
##     sign
```

```
##
## Attaching package: 'Ecdat'

## The following object is masked from 'package:datasets':
##
##     Orange
```

```
data(Housing)
library(vip)

housing <- Housing

dim(housing)
```

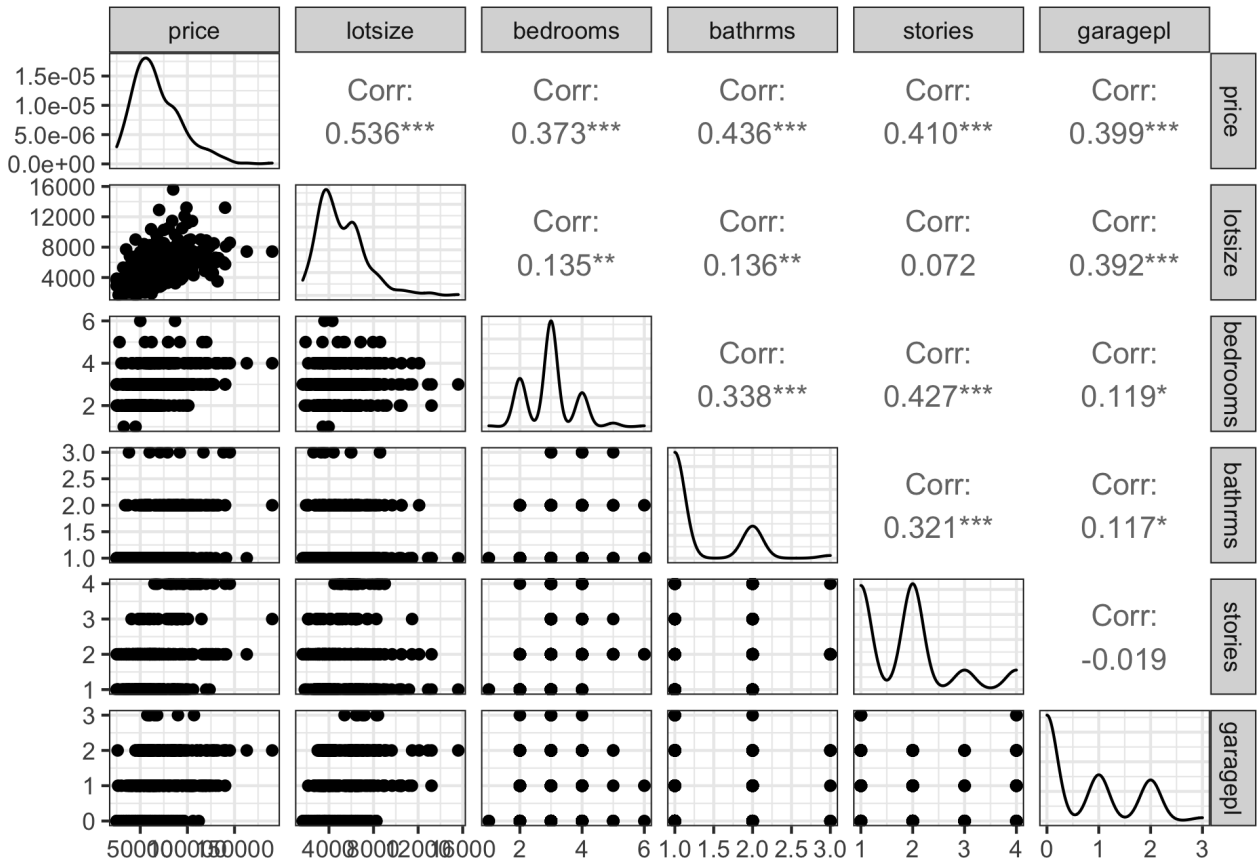
```
## [1] 546  12
```

```
# semente aleatoria

set.seed(9890, kind = "Mersenne-Twister", normal.kind = "Inversion")

# 77% dos dados como treino

housing_split <- initial_split(housing, prop = .77)

# criar os conjuntos de dados de treino e teste

housing_treino <- training(housing_split)

housing_teste  <- testing(housing_split)
```

Questão 12

(*05 pontos*) Utilizando o conjunto de treino, crie gráficos de dispersão e calcule as correlações duas a duas entre todas as variáveis quantitativas do conjunto de dados. Qual variável possui a maior correlação com o preço do imóvel? Como é possível justificar essa relação? Qual variável possui a menor correlação (isto é, mais próxima de zero) com o preço do imóvel? Como é possível justificar essa relação?

```
ggpairs(housing_treino %>%
          select(where(is.numeric)))
```



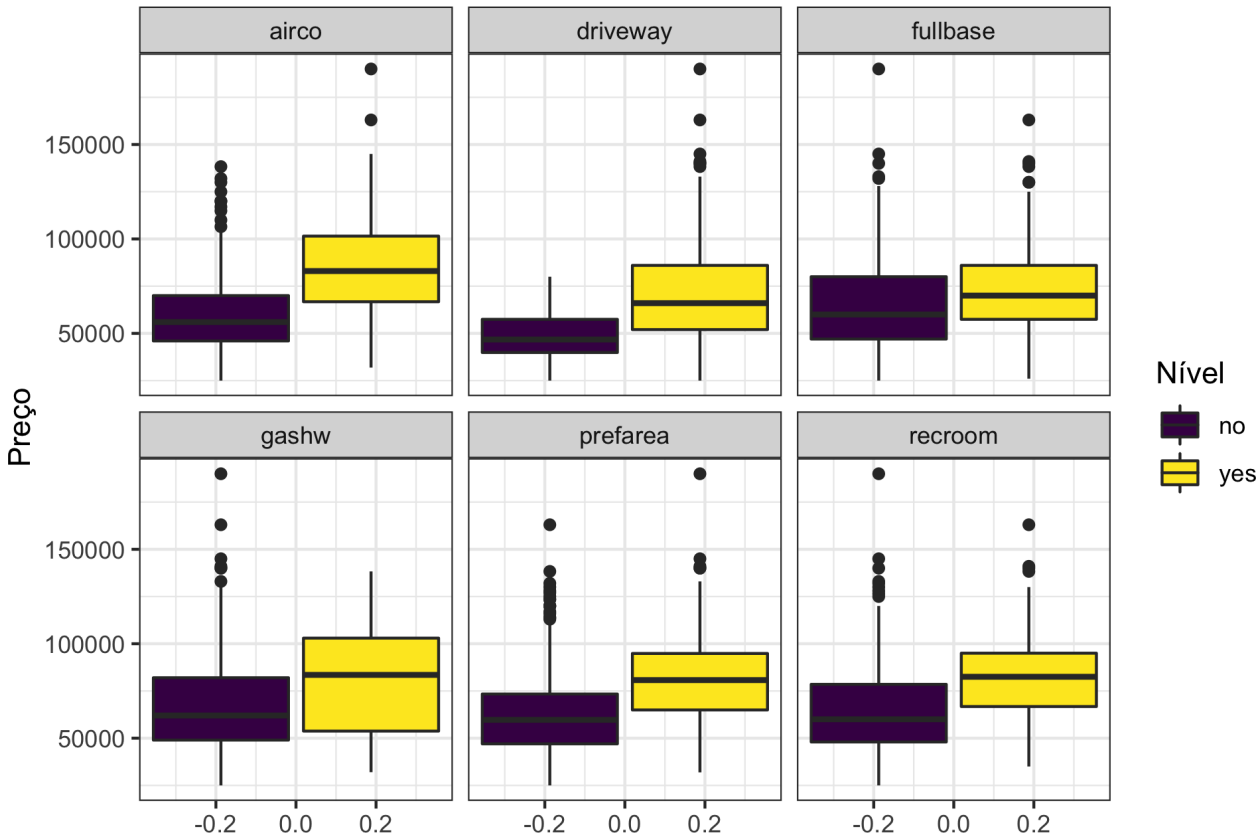
A variável que apresenta maior correlação com o preço do imóvel é `lotsize`. É algo que pode fazer sentido, posto que `lotsize` diz respeito ao tamanho do terreno onde o imóvel está construído. Quanto maior o terreno, maior tende a ser o imóvel e, quanto maior o imóvel, mais caro ele é.

A variável que apresenta menor correlação com o preço do imóvel é `garagepl`. Uma explicação possível para a pequena correlação é que os imóveis estão localizados em uma cidade com boa cobertura de transporte público, e portanto o número de vagas de garagem é irrelevante na hora de precificar um imóvel.

Questão 13

(*05 pontos*) Ainda com o conjunto de treino, crie boxplots comparando o preço dos imóveis de acordo com cada uma das variáveis categóricas preditoras. Qual dessas variáveis parece gerar a maior diferença entre sua ausência e a sua presença?

```
housing_treino %>%
  select(!where(is.numeric), price) %>%
  pivot_longer(-price) %>%
  ggplot(., aes(fill = value)) +
  geom_boxplot(aes(y = price)) +
  facet_wrap(~ name) +
  labs(x = "", y = "Preço", fill = "Nível") +
  scale_fill_viridis_d()
```



A variável `airco` parece gerar a maior diferença entre sua ausência e a sua presença.

Questão 14

(05 pontos) Para pré-processar as variáveis preditoras, escreva uma receita que

i) Crie variáveis dummy a partir das variáveis categóricas;

ii) Normalize as variáveis quantitativas usando a transformação de Yeo Johnson;

iii) Use center e scale para deixar todas as variáveis na mesma escala.

Não é necessário realizar nenhum outro tipo de pré-processamento para essa análise. Aplique as transformações nos conjuntos de treino e teste.

```
housing_rec <-
  recipe(price ~ .,
         data = housing_treino) %>%
  # criar variaveis dummy
  step_dummy(driveway,
             recroom,
             fullbase,
             gashw,
             airco,
             prefarea) %>%
  # tranformar variaveis numericas usando yeo johnson
  step_YeoJohnson(lotsize,
```

```
                  bedrooms,
                  bathrms,
                  stories,
                  garagepl) %>%
  # center/scale
  step_center(-price) %>%
  step_scale(-price) %>%
  # funcao para aplicar a transformacao aos dados
  prep()

# aplicar a transformacao aos dados

housing_treino_t <- juice(housing_rec)

# preparar o conjunto de teste

housing_teste_t <- bake(housing_rec,
                        new_data = housing_teste)
```

Questão 15

(05 pontos) Utilize a semente 1702 para criar a validação cruzada com 7 grupos.

```
# definicao da validacao cruzada

set.seed(1702, kind = "Mersenne-Twister", normal.kind = "Inversion")

housing_treino_cv <- vfold_cv(housing_treino_t, v = 7)
```

Questão 16

(05 pontos) Crie grids de procura para os hiperparâmetros dos métodos SVM linear e random forest. Encontre o melhor valor de custo para o SVM entre os valores 2^{-10} e 2^{10}. Para o random forest, escolha quatro valores diferentes para `mtry` entre 1 e o máximo permitido, deixe `trees` com o valor único de 1000 e defina oito valores para `min_n` entre 10 e 80.

```
# definicao do tuning - svm

housing_svm_poly_tune  <-
  svm_poly(
    cost = tune(),
    degree = 1) %>%
  set_engine("kernlab") %>%
  set_mode("regression")

# grid de procura

housing_svm_poly_grid <- grid_regular(cost (range(-10, 10)),
                                      levels = 11)

# definicao do tuning - rf
```

```r
housing_rf_tune <-
  rand_forest(
    mtry = tune(),
    trees = 1000,
    min_n = tune()
  ) %>%
  set_mode("regression") %>%
  set_engine("ranger", importance = "impurity")

# grid de procura

housing_rf_grid <- grid_regular(mtry(range(1, 11)),
                                min_n(range(10, 80)),
                                levels = c(11, 4))
```

Questão 17

(*05 pontos*) Determine qual dos métodos utilizados apresentou o melhor resultado no conjunto de treino, considerando o $RMSE$.

```r
# workflow - svm

housing_svm_poly_tune_wflow <-
  workflow() %>%
  add_model(housing_svm_poly_tune) %>%
  add_formula(price ~ .)

# avaliacao do modelo - svm

housing_svm_poly_fit_tune <-
  housing_svm_poly_tune_wflow %>%
  tune_grid(
    resamples = housing_treino_cv,
    grid = housing_svm_poly_grid
  )

# workflow - rf

housing_rf_tune_wflow <-
  workflow() %>%
  add_model(housing_rf_tune) %>%
  add_formula(price ~ .)

# avaliacao do modelo - svm

housing_rf_fit_tune <-
  housing_rf_tune_wflow %>%
  tune_grid(
    resamples = housing_treino_cv,
    grid = housing_rf_grid
  )
```

```r
# melhor modelo - svm

housing_svm_poly_fit_tune %>%
  show_best("rmse", n = 1)
```

```
## # A tibble: 1 x 7
##     cost .metric .estimator   mean     n std_err .config
##    <dbl> <chr>   <chr>       <dbl> <int>   <dbl> <chr>
## 1 0.0156 rmse    standard   15463.     7    867. Preprocessor1_Model03
```

```r
# melhor modelo - rf

housing_rf_fit_tune %>%
  show_best("rmse", n = 1)
```

```
## # A tibble: 1 x 8
##    mtry min_n .metric .estimator   mean     n std_err .config
##   <int> <int> <chr>   <chr>       <dbl> <int>   <dbl> <chr>
## 1     3    10 rmse    standard   15622.     7    977. Preprocessor1_Model03
```

Como o $RMSE$ para o SVM Linear é 15463, o Random Forest apresentou um resultado ligeiramente melhor, com 15622.

Questão 18

(*05 pontos*) Houve sobreajuste em algum dos algoritmos utilizados? Justifique.

```r
# selecao e ajuste do melhor modelo - svm

housing_svm_poly_best <-
  housing_svm_poly_fit_tune %>%
  select_best("rmse")

housing_svm_poly_final <-
  housing_svm_poly_tune_wflow %>%
  finalize_workflow(housing_svm_poly_best)

housing_svm_poly_final <- fit(housing_svm_poly_final, housing_treino_t)

# selecao e ajuste do melhor modelo - rf

housing_rf_best <-
  housing_rf_fit_tune %>%
  select_best("rmse")

housing_rf_final <-
  housing_rf_tune_wflow %>%
  finalize_workflow(housing_rf_best)

housing_rf_final <- fit(housing_rf_final, housing_treino_t)
```

```r
# verificando sobreajuste

resultado_svm_poly <-
  housing_teste_t %>%
  bind_cols(predict(housing_svm_poly_final, housing_teste_t) %>%
              rename(predicao_svm_poly = .pred))

resultado_rf <-
  housing_teste_t %>%
  bind_cols(predict(housing_rf_final, housing_teste_t) %>%
              rename(predicao_rf = .pred))

# metricas finais no conjunto de teste

metrics(resultado_svm_poly,
        truth = price,
        estimate = predicao_svm_poly)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard   17423.
## 2 rsq     standard       0.705
## 3 mae     standard   12039.
```

```r
metrics(resultado_rf,
        truth = price,
        estimate = predicao_rf)
```

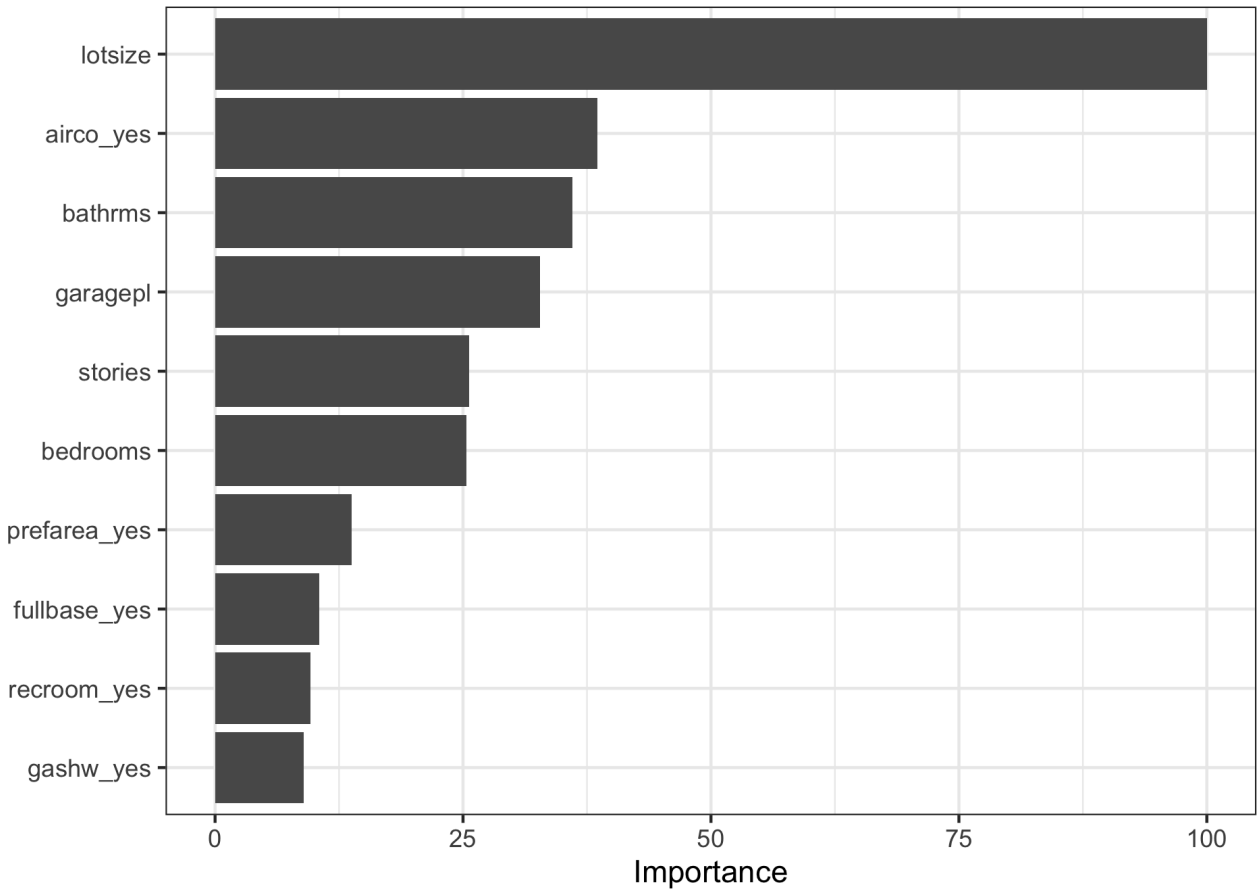
```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard   18444.
## 2 rsq     standard       0.648
## 3 mae     standard   12366.
```

Os valores das métricas nos conjuntos de treino e teste são similares para ambos os algoritmos, então não temos motivos para suspeitar de sobreajuste.

Questão 19

(*05 pontos*) Segundo o random forest, qual é a variável mais importante no modelo ajustado? Intuitivamente, esse resultado faz sentido?

```r
housing_rf_final %>%
  pull_workflow_fit() %>%
  vip(scale = TRUE)
```



A variável mais importante é `lotsize`. Como essa é a variável referente ao tamanho do terreno do imóvel, faz sentido que seja a que mais influencie em seu preço.

Questão 20

(*05 pontos*) O resultado obtido com a melhor modelagem é bom o suficiente, na sua opinião? Utilize argumentos numéricos e gráficos para justificar a sua resposta.

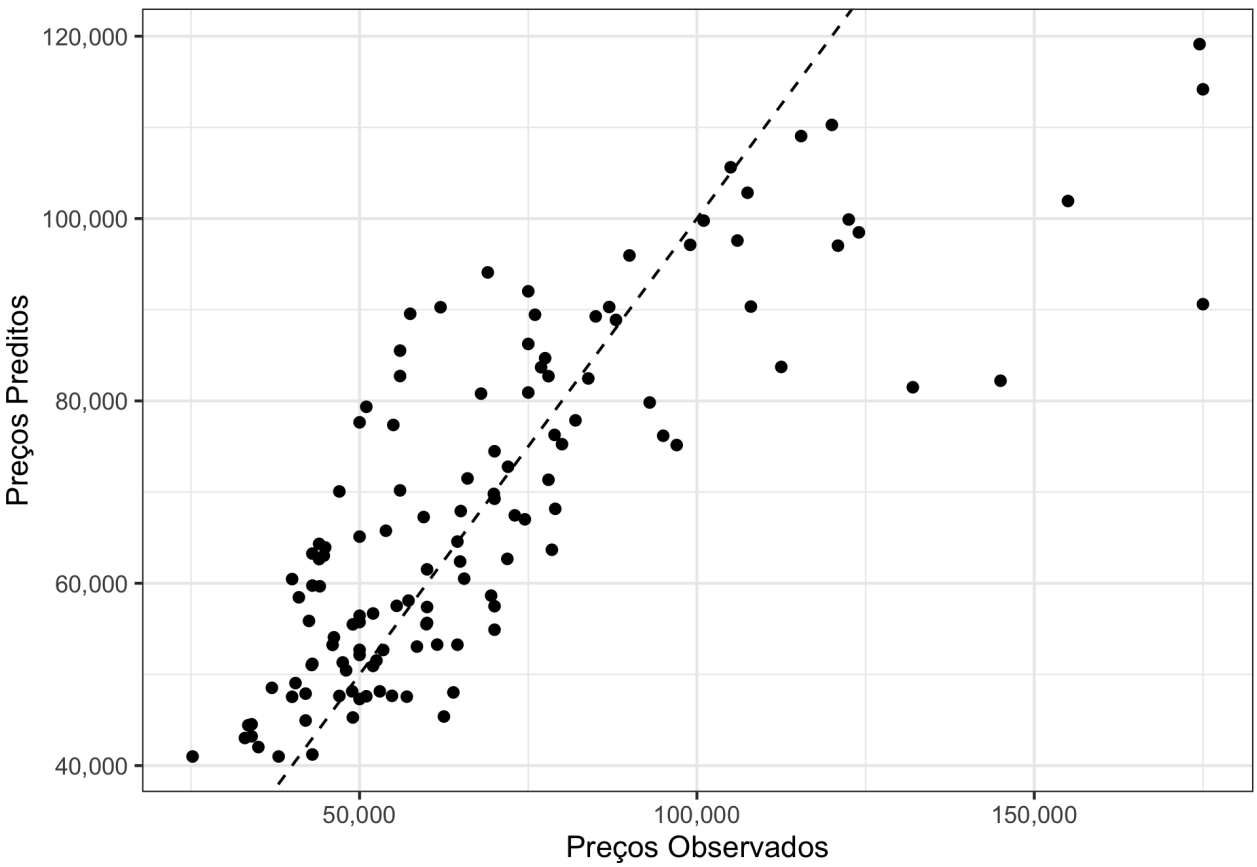
```
# rmse treino

housing_rf_fit_tune %>%
  show_best("rmse", n = 1)
```

```
## # A tibble: 1 x 8
##    mtry min_n .metric .estimator   mean     n std_err .config
##   <int> <int> <chr>   <chr>       <dbl> <int>   <dbl> <chr>
## 1     3    10 rmse    standard   15622.     7    977. Preprocessor1_Model03
```

```
# metricas teste

metrics(resultado_rf,
        truth = price,
        estimate = predicao_rf)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard   18444.
## 2 rsq     standard       0.648
## 3 mae     standard   12366.
```

```
# valores preditos vs observados

ggplot(resultado_rf, aes(x = price, y = predicao_rf)) +
  geom_point() +
  geom_abline(linetype = "dashed") +
  labs(x = "Preços Observados", y = "Preços Preditos") +
  scale_x_continuous(labels = comma) +
  scale_y_continuous(labels = comma)
```



A modelagem utilizando random forest não apresentou sobreajuste. Portanto, o modelo está bem ajustado aos dados.

Além disso, ao tomarmos o valor do RMSE no conjunto de treino, temos 15621.52. Esse valor, ao ser comparado com a média da variável `price` (67991.58), nos diz que o erro obtido é equivalente a 22.98% desse valor.

Por fim, embora os valores dos imóveis mais caros não tenham sido bem modelados, o gráfico de preços preditos versus preços observados apresenta alguma linearidade.